Day 24: Delete Node in Linked List

📌 1. Concept (Basic Understanding)

Linked List me delete karna matlab:

👉 Kisi node ko list se hata dena
👉 Uska reference tod dena

💡 Important Cases:

Delete at beginning

Delete at end

Delete by value / position

🔹 2. Delete at Beginning

🧠 Logic:

👉 Head ko next node pe move kar do

head = head.next;

📊 Dry Run:

List:
10 → 20 → 30 → 40

Delete first node:

Step:

head = 10

head = head.next → 20

Result:
20 → 30 → 40

✅ Code (Java)

```
1. public void deleteAtBeginning() {
    if (head == null) {
        System.out.println("List is empty");
        return;
    }

    head = head.next;
}
2.
```

🔹 3. Delete at End

🧠 Logic:

👉 Last node tak jao
👉 Second last node ka next = null karo

📊 Dry Run:

List:
10 → 20 → 30 → 40

Steps:

temp = head

temp.next.next == null → (30 is second last)

temp.next = null

Result:
10 → 20 → 30

✅ Code

```
1. public void deleteAtEnd() {
    if (head == null) {
        System.out.println("List is empty");
        return;
    }

    if (head.next == null) {
        head = null;
        return;
    }

    Node temp = head;

    while (temp.next.next != null) {
        temp = temp.next;
    }

    temp.next = null;
}
2.
```

🔹 4. Delete by Value

🧠 Logic:

👉 Jis value ko delete karna hai usko search karo
👉 Uske previous node ka next change karo

📊 Dry Run:

List:
10 → 20 → 30 → 40

Delete 30:

Steps:

temp = 20 (previous node)

temp.next = temp.next.next

Result:
10 → 20 → 40

✅ Code

```
public void deleteByValue(int key) {
    if (head == null) {
        System.out.println("List is empty");
        return;
    }

    if (head.data == key) {
        head = head.next;
        return;
    }

    Node temp = head;

    while (temp.next != null && temp.next.data != key) {
        temp = temp.next;
    }

    if (temp.next == null) {
        System.out.println("Value not found");
        return;
    }

    temp.next = temp.next.next;
}
```

🔥 5. Full Combined Code (Interview Ready)

```
 1. package Day_24;
 2.
 3.     class LinkedList {
 4.     Node head;
 5.
 6.     class Node {
 7.         int data;
 8.         Node next;
 9.
10.         Node(int data) {
11.             this.data = data;
12.             this.next = null;
13.         }
14.     }
15.
16.     // Insert at Beginning
17.     public void insertAtBeginning(int data) {
18.         Node newNode = new Node(data);
19.         newNode.next = head;
20.         head = newNode;
21.     }
22.
23.     // Insert at End
24.     public void insertAtEnd(int data) {
25.         Node newNode = new Node(data);
26.
27.         if (head == null) {
28.             head = newNode;
29.             return;
30.         }
31.
32.         Node temp = head;
33.         while (temp.next != null) {
34.             temp = temp.next;
```

```
        }

        temp.next = newNode;
    }

    // Delete at Beginning
    public void deleteAtBeginning() {
        if (head == null) return;
        head = head.next;
    }

    // Delete at End
    public void deleteAtEnd() {
        if (head == null) return;

        if (head.next == null) {
            head = null;
            return;
        }

        Node temp = head;
        while (temp.next.next != null) {
            temp = temp.next;
        }

        temp.next = null;
    }

    // Delete by Value
    public void deleteByValue(int key) {
        if (head == null) return;

        if (head.data == key) {
            head = head.next;
            return;
        }

        Node temp = head;

        while (temp.next != null && temp.next.data != key) {
            temp = temp.next;
        }

        if (temp.next != null) {
            temp.next = temp.next.next;
        }
    }

    // Display
    public void display() {
        Node temp = head;
        while (temp != null) {
            System.out.print(temp.data + " -> ");
            temp = temp.next;
        }
        System.out.println("null");
    }
}
public class LinkedlistDelete {
    public static void main(String[] args) {

        LinkedList list = new LinkedList();

        list.insertAtBeginning(10);
        list.insertAtEnd(20);
        list.insertAtEnd(30);
        list.display();

        System.out.println("Delete at Beginning");
        list.deleteAtBeginning();
```

```
        list.display();

        System.out.println("Delete at End");
        list.deleteAtEnd();
        list.display();

        System.out.println("Delete by Value (20)");
        list.deleteByValue(20);
        list.display();
    }
}
```

Time Complexity

Operation	Time
Beginning	O(1)
End	O(n)
By Value	O(n)